

Evaluating Python Asyncio for an Application Server Herd

Computer Science 131 - Programming Languages
University of California Los Angeles
Bhavesh Pathak

Abstract

Python's asyncio framework is a networking library intended for asynchronous communication across multiple interconnected servers. This paper investigates its merit as a potential candidate for making a platform of interconnected servers that is like Wikimedia but updates to articles happen far more often, access is required via multiple protocols and clients are more mobile. In comparison to traditional centralized application-server architectures, this will be implemented using an application server herd. An application server herd sends updates directly between servers, thus removing the need for a central server/database. To investigate the suitability of asyncio for this environment, I implemented a prototype consisting of five interconnected servers. The prototype accepts client location updates, propagates them throughout the server herd using an asynchronous flooding algorithm, and retrieves nearby location information through the Google Places API. Results from the prototype indicate that asyncio significantly simplifies the development of concurrent network applications using asynchronous I/O. However, Python's dynamic type system, global interpreter lock, and cooperative scheduling model introduce limitations compared to statically typed, multithreaded Java-based solutions. Despite these limitations, asyncio is well suited for I/O-bound distributed services such as the application server herd studied here. This leads me to conclude that asyncio is indeed a practical framework for this scenario.

1 Introduction

Wikipedia relies on a centralized application server architecture built on a rigid stack: Apache and NGINX for HTTP serving, MariaDB for relational storage, Memcached for caching, and PHP+JavaScript for application logic. While this architecture is well-proven for static encyclopedia-style content, it is poorly suited to a news-oriented service that updates frequently, has to deal with mobile clients and must support access across multiple protocols. The centralized

PHP+JavaScript application server becomes a bottleneck under these conditions, both in terms of software extensibility and response time. This is because in centralized applications, everything must be sent through the central server or database, so it cannot handle frequent updates from multiple locations. This would not be a problem for a distributed application though, because the server requests are spread out among servers by region so the bottleneck is reduced. This report investigates a particular distributed architecture called an application server herd, in which multiple application servers communicate directly with one another for rapidly-evolving data such as GPS locations and ephemeral media. Location updates posted to any one server propagate to the rest of the herd through inter-server messaging. The specific question this report addresses is whether Python's asyncio library is a suitable framework for implementing such a herd. Asyncio's event-driven, cooperative concurrency model is a natural candidate for I/O-bound inter-server communication, but practical concerns remain around Python's dynamic type system, memory management, garbage collection, and inability to achieve true thread-level parallelism due to the Global Interpreter Lock. To evaluate these concerns concretely, a prototype herd of five interconnected servers was implemented, capable of accepting client location updates, propagating them via a flooding algorithm, and servicing nearby place queries through the Google Places API. The remainder of this report describes the prototype, analyzes asyncio's suitability along each of the dimensions raised, compares it to Java and Node.js as alternatives, and concludes with a recommendation.

1.1 Prototype Design

The prototype consists of five asyncio-based servers named Bailey, Bona, Campbell, Clark, and Jaquez, communicating according to a fixed bidirectional topology. Clark talks with Bona and Jaquez; Campbell talks with Bailey, Bona, and Jaquez; and Bona talks with Bailey. Each server is launched as a standalone Python process via `python3 server.py <name>` and listens on a statically assigned port. Incoming connections

are handled by `handle_client`, an async coroutine registered with `asyncio.start_server` that reads one line per connection and dispatches on the command name. Three commands are supported. IAMAT accepts a client ID, an ISO 6709 location string, and a POSIX timestamp; the server records the client's location in an in-memory dictionary, responds with an AT message containing the time differential between the server's receipt time and the client's timestamp, and floods the AT message to all neighbours. WHATSAT accepts a client ID, a radius in kilometres, and a result count bound; the server retrieves the stored location for that client, queries the Google Places API for nearby places, and responds with the AT message followed by a JSON array of results. AT messages received from neighbours trigger a flooding update: if the incoming record is newer than the locally stored one, the server updates its state and re-floods to all neighbours except the sender, preventing infinite loops. Servers that cannot reach a neighbour log a warning and continue, satisfying the spec's requirement that the herd remain operational when individual servers go down.

1.2 Google Places API

When a WHATSAT query arrives, the server calls `query_places`, an async coroutine that issues an HTTP POST to the Google Places Nearby Search endpoint at <https://places.googleapis.com/v1/places:searchNearby> using the `aiohttp` library. The request body specifies a circular search region centered on the client's stored latitude and longitude with the requested radius converted from kilometres to metres, and a `maxResultCount` capped at 20 per the API's limit. Authentication is provided via the `X-Goog-Api-Key` header, and the `X-Goog-FieldMask` header restricts the response to the fields relevant to the spec: name, ID, location, rating, types, and photos. Because `aiohttp` is fully async, the HTTP request yields control to the event loop while waiting for Google's response, allowing the server to continue handling other connections in the interim rather than blocking. The JSON response is deserialized, the places array extracted, and the result serialized back to a string using `json.dumps` with two-space indentation. A post-processing step collapses any sequences of two or more adjacent newlines into a single newline and strips trailing newlines, as required by the spec. The cleaned JSON is appended to the AT line separated by a newline, and the combined response is terminated with two newlines before being written to the client.

2 Recommendation

I recommend using `asyncio`, with some important caveats. In most other circumstances, other approaches, namely a Java-based approach would be more effective, but these circumstances are ideal for `asyncio`. The workload described — high-frequency updates, many concurrent mobile connections, ac-

cess via multiple different protocols, — is matched very well to `asyncio`'s strengths. `Asyncio`'s event loop means it scales well as the number of concurrent connections increase because there's no memory overhead for thread computations. It has proven to outperform traditional multithreading when dealing with I/O bottlenecks because the CPU can do other tasks while waiting on requests or responses.

2.1 Caveats

`Asyncio` is not designed for CPU-intensive work like media processing, machine learning or any other application where there are large quantities of independent computations. Java and other programming languages' multithreading computer models are much better at this kind of problem because their multiple threads can compute multiple things at once. `Asyncio`'s efficiency is not derived from multithreading, but rather from minimizing time wasted waiting for responses. Python's dynamic type system requires more skill and effort to use without introducing bugs. This puts the burden of writing type-safe code on the programmer, whereas Java would prevent this by refusing to compile code that isn't type-safe. `Asyncio`'s event loop model may also be vulnerable to bugs like missing `await` keywords or infinite loops. These bugs will compile without an error message but then keep the code stuck forever - occupying the only "thread" and thus preventing any other work from getting done.

3 How Java Compares

Java is one of the oldest, most traditional and most popularly used languages for this type of application, so it makes sense to use Java as a reference point for comparison. Java's static type system catches type errors at compile time rather than runtime, its preemptive threading model enables true parallelism across multiple cores, and the garbage collector is more mature and tunable for predictable latency under heavy load. However, this comes at the cost of Java being significantly less user-friendly, with its syntax being harder to learn and debug, having slower development time and introducing potential data races into the code if not handled carefully. For large engineering teams who intend to build and maintain a long-lasting product, Java's type system, object-orientation and overall verbosity is very useful because it helps simplify the architecture of very complicated codebases. By contrast, I was able to build a small, simple and working prototype by myself relatively quickly in `asyncio`.

So for this specific workload, however, `asyncio` compares favorably. The server herd is almost entirely I/O-bound, which is precisely the scenario `asyncio` is designed for. The flooding logic, connection handling, and external API calls in the prototype would require considerably more boilerplate in Java. In addition, multi-threaded Java code would have to be synchronized in order to prevent data races. `Asyncio`'s cooperative

scheduling eliminates this synchronization burden entirely, since data access patterns are predictable at await boundaries. Development time is also meaningfully lower in Python for a small team iterating quickly on a prototype. My overall assessment is that Java is the safer choice for a large production system with a sizeable engineering team, while asyncio is the more pragmatic choice for a small team building an I/O-bound service that needs to evolve quickly — which matches the scenario described here.

3.1 Type Checking

A very valid concern in regards to using Python for this application is that of type checking. Because Python is dynamically typed, it checks the types of variables at runtime, which means type errors are possible while a server is deployed. In this particular context, this is inferior to Java, whose static typing system checks for type errors at compile time and thus catches these errors before deployment. Python cannot give this kind of safety guarantee, however, some careful defensive programming procedures can make up for it. For example, in my prototype, I use casting to prevent type errors during runtime. This can be seen in lines of code such as `"float(timestamp)"` or `"int(max_results)"`. Another risk of using asyncio instead of Java in this context is that forgetting to await a coroutine is a silent bug - it produces no error message and compiles while doing absolutely nothing. This class of bug does not exist in Java's thread-based concurrency model, where method calls execute immediately without a separate scheduling step. Overall, Python's type safety is weaker than Java's, but with disciplined programming practices can make it manageable for an I/O-bound service like this one.

3.2 Memory Management

Asyncio, and Python in general, are suited to this type of application because of Python's memory management system. In particular, Python's automated garbage collection frees memory that is no longer being used. This is useful because it frees the programmer from having to manually allocate and deallocate memory, as in languages like C/C++. However, it is not a good metric for comparison with Java, as Java also has a similar approach to memory management and also performs automated garbage collection. If anything, Java's approach to garbage collection is more versatile because it allows the programmer to modify the garbage collector by setting parameters.

3.3 Multithreading

Achieving true parallelism in Python is impossible because of the global interpreter lock (GIL). This is a mechanism that prevents more than one thread from executing Python bytecode at the same time, even on multicore hardware. There

can be multiple threads operating independently, but these threads cannot be operating concurrently. An upside of this is that there is no need to worry about race conditions in Python - because only one thread can be running at a time, Python guarantees data integrity because threads cannot interact.

By comparison, Java's multithreading model uses OS-level threads that can run concurrently, substantially reducing compute time in cases of large, CPU-heavy computations. However, there is significant overhead for creating threads and context switching, so it may not necessarily be faster in this particular context. Java's parallelism also comes at the risk of potentially compromising data integrity, as these different threads can interact with each other in unpredictable ways and introduce race conditions. Although these risks can be mitigated by using keywords such as `Synchronized`, this would still require significant caution and attention from the developers.

For a server handling many clients simultaneously, it sounds like parallelism would be greatly beneficial. However, the server herd is almost entirely I/O-bound: servers spend the majority of their time waiting on network reads, flood connections to neighbours, and Google Places API responses. The GIL is released during I/O operations, so asyncio can freely interleave coroutines during these waits without being blocked by it. Parallelism would be most useful for complex, CPU-heavy computations, of which we can expect very little in this context.

One more factor to consider is vulnerability. Asyncio's single event loop model of computation means it can be held up by misbehaving coroutines, e.g. a Places API call that hangs indefinitely. In this scenario, the entire event loop is blocked and unable to continue performing any more operations. Java does not have this same weakness because its multithreading model means that other threads can keep going even if one gets hung up.

3.4 Node.js

Node.js is the closest architectural analogue to asyncio among mainstream alternatives, and the comparison is informative precisely because the two are so similar. Both use a single-threaded event loop with non-blocking I/O to handle many concurrent connections without the overhead of thread-per-connection models. Both adopt cooperative concurrency, exposing it to the programmer through `async/await` syntax that asyncio introduced in Python 3.5 and Node.js adopted around the same time after an earlier callback-based model. For the application server herd described here, either platform would be architecturally appropriate — the event-driven model maps naturally onto the flooding algorithm, and both handle the I/O-bound workload of location updates and external API calls without difficulty.

The differences are mostly practical rather than architectural. Node.js uses the V8 JavaScript engine and the libuv

event loop library, which is mature, heavily optimized for networked I/O, and backed by a large ecosystem of real-time networking libraries such as Socket.io that would be directly relevant to a mobile news service. Python’s `asyncio` event loop is newer and its ecosystem smaller, though libraries like `aiohttp` cover the necessary ground for this application. One meaningful difference is that `asyncio` makes concurrency more explicit: coroutines must be explicitly declared with `async` and awaited with `await`, making it clear at every call site where a context switch can occur. Node.js’s historical reliance on callbacks, and its later adoption of promises and `async/await`, has produced a more fragmented ecosystem where older interfaces mix concurrency styles. For a team already working in Python, `asyncio` is the natural choice; for a team comfortable in JavaScript, Node.js offers a richer real-time ecosystem with comparable architectural trade-offs. Neither has a decisive technical advantage for this specific workload.

4 Ease of Use

One of `asyncio`’s most practical advantages is how naturally it maps onto the application server herd problem. Each server’s responsibilities — accepting connections, parsing messages, updating state, and forwarding updates to neighbours — translate directly into coroutines that yield control at `await` points, allowing the event loop to interleave them without explicit thread management. In the prototype, the core flooding logic requires only a handful of lines: `asyncio.open_connection` opens a TCP connection to a neighbour, `writer.write` sends the message, and `await writer.drain` yields until the write completes. Compared to a Java implementation requiring explicit `ExecutorService` configuration, `Runnable` definitions, and synchronized access to shared state, the `asyncio` version is substantially more concise. The main difficulty is conceptual rather than syntactic: programmers must internalize that `await` is the only point at which another coroutine can run, and that any blocking call without `await` stalls the entire event loop. In practice this means standard library functions that perform I/O must be replaced with their `asyncio`-aware equivalents, and third-party libraries must be vetted for `async` compatibility. The learning curve is moderate for programmers already familiar with Python but steeper for those coming from synchronous or thread-based backgrounds. Overall, `asyncio` strikes a reasonable balance between expressiveness and explicitness for this class of application.

5 Performance Implications

`Asyncio`’s performance characteristics are well matched to the server herd workload. Because the herd is I/O-bound — servers spend most of their time waiting on network reads, neighbour connections, and Google Places responses — `asyncio`’s single-threaded event loop can keep the CPU

productively occupied by interleaving coroutines during these waits, without the context-switching overhead of a thread-per-connection model. For a high-connection-count mobile service where many clients are simultaneously broadcasting GPS locations, this is a meaningful advantage. The cost of `asyncio`’s model appears when work becomes CPU-bound: since the GIL prevents parallel bytecode execution across threads, and `asyncio` uses only one thread, any CPU-intensive operation blocks all other coroutines for its duration. For the prototype this is not a concern, but a production service that added article ranking, media transcoding, or natural language processing would need to offload that work via `asyncio.run_in_executor` to a thread or process pool, adding architectural complexity. A further consideration is that Python itself is slower than Java at raw execution, though for I/O-bound workloads this gap is largely irrelevant since the bottleneck is network latency rather than CPU throughput. In summary, `asyncio` delivers competitive performance for this application’s actual workload, with the caveat that CPU-bound extensions would require additional design work.

6 Relevance of Python 3.9

The prototype relies on several `asyncio` features that either were introduced or significantly stabilized in Python 3.9 and later, and using an older version would introduce meaningful friction. The most important is `asyncio.run`, introduced in Python 3.7 and made non-provisional in 3.10, which provides a clean single-call entry point for running a coroutine as the top-level event loop. Prior to this, the idiomatic pattern required manually obtaining a loop via `asyncio.get_event_loop`, calling `loop.run_until_complete`, and closing the loop explicitly — boilerplate that obscures intent and is easy to get wrong. Python 3.9 also introduced `python3 -m asyncio`, an interactive REPL that allows coroutines to be awaited directly at the prompt, which significantly accelerates debugging and experimentation. Type annotation syntax improved substantially in 3.9 as well, allowing built-in collections like `list` and `dict` to be used directly as generic types rather than requiring imports from the `typing` module, which simplifies annotating `async` function signatures. While it is technically possible to implement the prototype on Python 3.6 or 3.7, the resulting code would be more verbose and less readable, and the lack of a stable `asyncio.run` would make the entry point more fragile. Given that Python 3.9 is now several years old and widely available, there is no practical reason to target an older version, and doing so would sacrifice real usability improvements for no meaningful benefit.

7 Conclusion

Based on the evidence from the implementation of the prototype and the analysis of Python presented in this report, I

deem `asyncio` to be a suitable framework for this application. Compared to a Java-based approach, there are three major areas of concern: the type system, memory management, and multithreading. I will concede that in these particular areas, a Java-based approach does outshine `asyncio`, but not by a significant amount. Python's dynamic type system is weaker than Java's compile-time checking, but disciplined use of type casting and other defensive programming procedures brings it close enough for a service of this scope. Both languages have approximately the same garbage collection system when it comes to memory management, although Java's is a little bit more flexible. Python's GIL prevents true parallelism that Java has, but it also provides a memory safety guarantee that Java doesn't. While Java may seem like the better choice in these 3 languages, `asyncio` is easier to use and better suited to the application.

The prototype demonstrates that the core functionality — location updates, inter-server flooding, and external API integration — can be implemented concisely and correctly, with the cooperative scheduling model making data access patterns straightforward to reason about. The primary caveats are that `asyncio`'s cooperative model requires programmer discipline around `await` usage and timeouts, the dynamic type system demands consistent use of static analysis tools to compensate for the lack of compile-time checking, and the prototype's unbounded `client_data` structure would require an eviction policy before production deployment. If the service were later extended to include CPU-intensive work such as media processing or article ranking, Python's GIL would become a genuine bottleneck and a Java-based or multiprocessing architecture would warrant reconsideration. For the described workload of a mobile-heavy, I/O-bound news service with frequently updated location data, however, `asyncio` offers a compelling combination of development simplicity, concise expression of concurrent logic, and sufficient performance — making it the practical choice for this application today.

References

- [1] Python Software Foundation. *asyncio — Asynchronous I/O*. Python 3.11.2 Documentation.
<https://docs.python.org/3.11/library/asyncio.html>
- [2] L. Ramalho. *Fluent Python*, 2nd ed. O'Reilly Media, 2021. ISBN 9781492056355.
- [3] aiohttp contributors. *aiohttp Documentation*.
<https://docs.aiohttp.org/>
- [4] Google. *Places API (New) — Nearby Search*. Google for Developers.
<https://developers.google.com/maps/documentation/places/>
- [5] Python Software Foundation. *PEP 703 — Making the Global Interpreter Lock Optional*.
<https://peps.python.org/pep-0703/>